

Day 114: DBSCAN Clustering — Beginner Introduction

1. Day number

Day 114

2. Topic name

Density-Based Clustering with DBSCAN

DBSCAN stands for:

| **Density-Based Spatial Clustering of Applications with Noise**

You do not need to memorize the full name yet. The important phrase is **density-based clustering**.

3. Connection

Yesterday, with **K-Means**, you learned that we usually decide the number of clusters first:

k = 2
k = 3
k = 4

K-Means then tries to divide the data into exactly that many groups.

Today, we will learn a different idea.

Instead of saying:

| "Create exactly 3 clusters."

DBSCAN asks:

| "Where are there dense groups of nearby points?"

It can also decide that some points do **not belong to any cluster**. These points can be treated as **noise or outliers**.

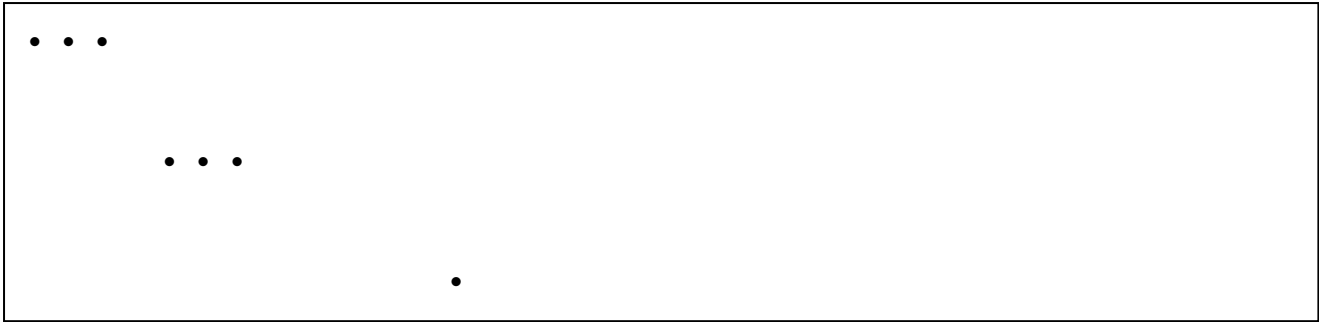
4. Important topics

Density

In simple terms, **density** means:

| How many data points are packed closely together?

Imagine these points:



There seem to be:

- one dense group on the left,
- another dense group in the middle,
- one isolated point on the right.

DBSCAN tries to discover patterns like this.

Neighborhood

A **neighborhood** means the area around a point.

Suppose we have one point:



X has several nearby points.

DBSCAN checks the neighborhood around X to see whether enough other points are close to it.

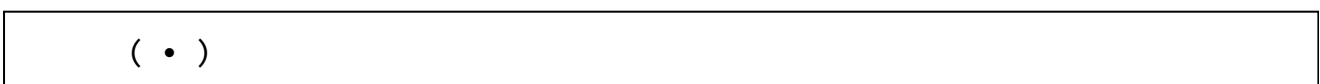
eps idea

eps controls approximately:

How close do points need to be to count as neighbors?

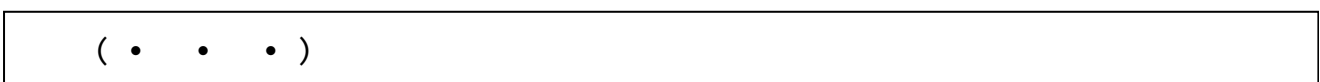
Think of drawing a small circle around every point.

Small eps:



Only very close points count as neighbors.

Larger eps:



More points may fall inside the neighborhood.

You do **not** need the mathematical distance formula today.

Minimum samples idea

DBSCAN also needs to know:

| How many nearby points are required before an area is considered dense?

In scikit-learn, this is commonly called:

`min_samples`

Conceptually:

`eps` → how close?

`min_samples` → how many nearby points are enough?

Together, these help DBSCAN decide whether a dense region exists.

Cluster

A **cluster** is a connected dense group of nearby points.

For example:

Cluster 1

• •
• • •
•

and:

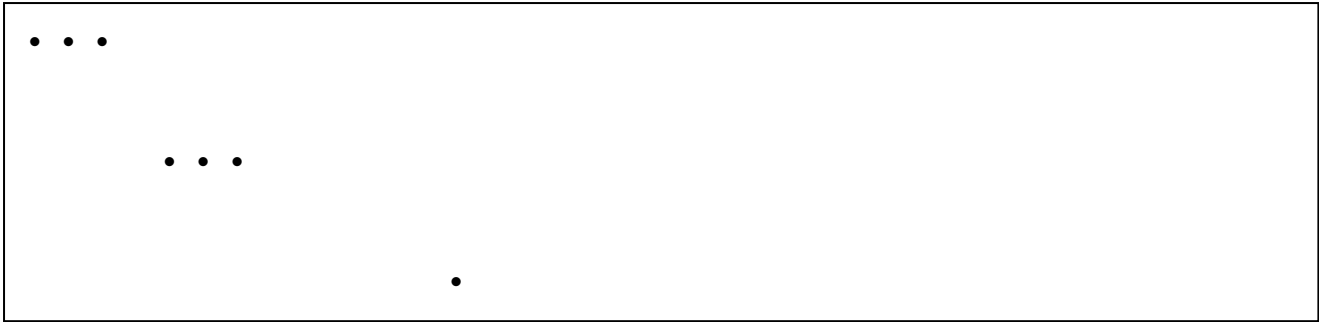
Cluster 2

• •
• • •

DBSCAN can discover these groups without requiring you to specify exactly how many clusters there should be.

Noise / outlier

Suppose we have:



The isolated point on the right may not have enough nearby neighbors.

DBSCAN may mark it as:

noise

In scikit-learn DBSCAN results, noise points commonly receive the cluster label:

-1

So you might see:

0
0
0
1
1
1
-1

Here, 0 and 1 represent two clusters, while -1 represents noise.

5. Foundational notes

DBSCAN is another **unsupervised learning** algorithm.

Like K-Means, it does not require known target labels.

We might have:

X = features

but no:

y = known answer

DBSCAN looks at how points are positioned relative to one another.

A useful mental model is:

Are several points close together?

↓

Yes

↓

Could they form a dense region?

↓

Yes

↓

Connect nearby dense regions

↓

Form a cluster

If a point is too isolated:

Isolated point

↓

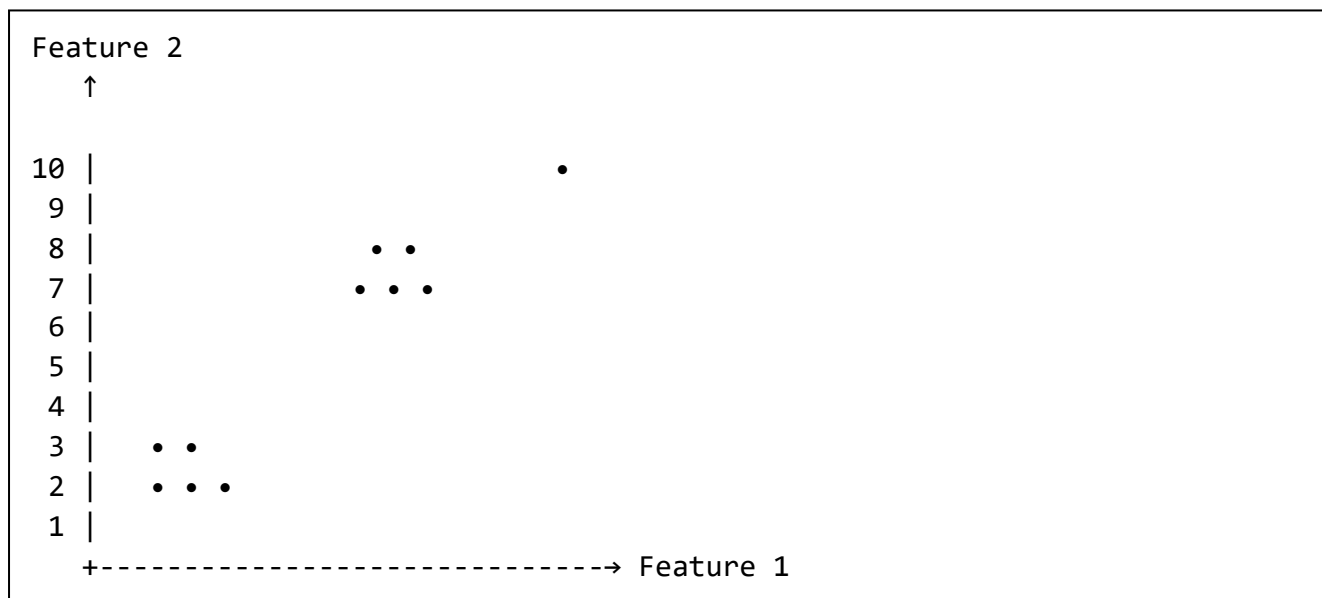
Not enough nearby points

↓

Possible noise

6. DBSCAN visually and intuitively

Imagine customers plotted according to two features:



Visually, you might notice:

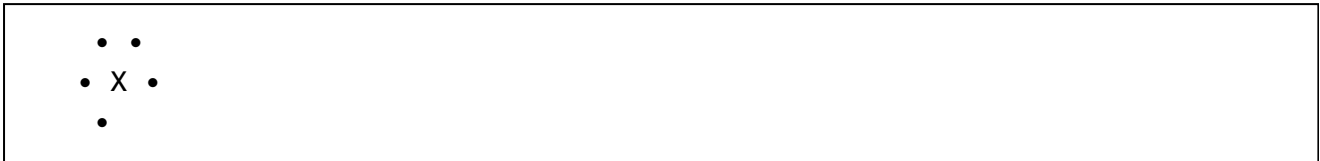
Group A → bottom-left points

Group B → middle-right points

Noise → isolated top-right point

DBSCAN starts examining points and their neighborhoods.

Suppose one point has enough nearby neighbors:



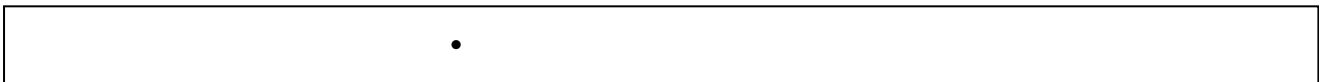
DBSCAN considers this a strong part of a dense region.

Nearby points can connect that region to more neighboring points:



The cluster can therefore grow.

Now consider:



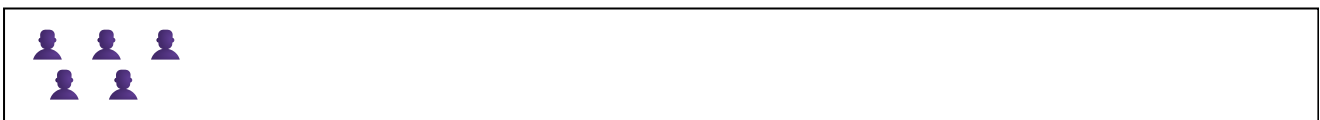
There are no close neighbors.

DBSCAN may leave this point outside the clusters and mark it as noise.

Simple DBSCAN mental model

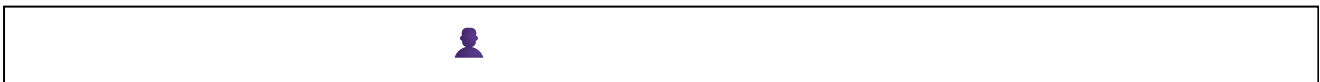
Think of a crowded room.

If several people are standing closely together:



you might naturally call them a group.

Someone standing far away:



might not belong to that group.

That is roughly the intuition behind DBSCAN.

7. DBSCAN vs K-Means

Idea	K-Means	DBSCAN
Unsupervised learning	Yes	Yes
Creates clusters	Yes	Yes
Must choose number of clusters first?	Usually yes (k)	No
Main idea	Distance to centroids	Dense neighborhoods
Uses centroids	Yes	No
Can explicitly mark noise	Not naturally	Yes
Important settings	k	eps, min_samples

K-Means thinking

```
"I want 3 groups."  
  ↓  
k = 3  
  ↓  
Find 3 centroids  
  ↓  
Assign every point
```

DBSCAN thinking

```
"Find regions where points are densely packed."  
  ↓  
Check neighborhoods  
  ↓  
Connect dense points  
  ↓  
Create clusters  
  ↓  
Leave isolated points as possible noise
```

One particularly important difference is that K-Means generally gives **every point a cluster**, while DBSCAN can say:

| "This point does not fit into a sufficiently dense group."

8. Easy example

Suppose we have these two-dimensional points:

A = (1, 1)
B = (1, 2)
C = (2, 1)

D = (8, 8)
E = (8, 9)
F = (9, 8)

G = (20, 2)

Visualize them roughly as:

D E
F

A B

C G

You can probably notice three patterns:

A, B, C → close together

D, E, F → close together

G → far away

A sensible DBSCAN configuration might therefore identify:

A, B, C → one cluster

D, E, F → another cluster

G → possible noise

The exact result depends on the selected values of `eps` and `min_samples`.

9. Problem statement

Consider these points:

Point	X	Y
A	1.0	1.0
B	1.2	1.1
C	1.1	1.4
D	6.0	6.0
E	6.2	6.1
F	6.1	6.3
G	12.0	2.0

Your task is to **conceptually** identify:

1. Which points appear close enough to form one dense group?
2. Is there another dense group?
3. Which point looks isolated?
4. Which point might therefore become noise?
5. What could happen if eps were made much larger?

You do not need to calculate exact distances.

10. Concepts used

This exercise uses:

- unsupervised learning,
- clustering,
- two-dimensional features,
- density,
- neighborhoods,
- eps,
- min_samples,
- clusters,
- noise,
- outliers.

The feature matrix might conceptually look like:

```
X = [  
    [1.0, 1.0],  
    [1.2, 1.1],  
    [1.1, 1.4],  
    [6.0, 6.0],  
    [6.2, 6.1],  
    [6.1, 6.3],  
    [12.0, 2.0]  
]
```

Notice again:

No predefined cluster labels are provided.

DBSCAN must discover the grouping from the points themselves.

11. Thought process

When approaching a beginner DBSCAN problem, think in this order.

Step 1: Look for nearby points

Ask:

Which points seem close to one another?

Do not think about cluster numbers yet.

Step 2: Look for dense areas

A couple of nearby points may not necessarily form a meaningful dense group.

Ask:

Are enough points packed into this area?

This is where the `min_samples` idea matters.

Step 3: Consider `eps`

Imagine a neighborhood around each point.

Ask:

Would nearby points fall inside this neighborhood?

If `eps` is tiny, very few points may count as neighbors.

If it is very large, many points may become connected.

Step 4: Connect dense neighboring areas

Nearby dense points can become part of the same cluster.

Conceptually:

```
dense point
  ↓
dense point
  ↓
dense point

→ cluster
```

Step 5: Inspect isolated points

Ask:

```
Does this point have enough neighbors?
```

If not, it could become:

```
noise
```

12. Beginner-friendly pseudocode

```
START

prepare numerical data points

choose an eps value

choose a minimum number of nearby samples

create DBSCAN model

fit the model using the data points

for each point:
    inspect its cluster label

if label is -1:
    treat point as noise

otherwise:
    identify the cluster it belongs to

compare the discovered groups

END
```

With scikit-learn, the workflow is conceptually:

```
from sklearn.cluster import DBSCAN

prepare X

create DBSCAN using:
    eps = ...
    min_samples = ...

fit DBSCAN on X

get cluster labels

inspect clusters and noise
```

You will often encounter:

```
model.labels_
```

after fitting the model.

You might get something conceptually like:

```
[0, 0, 0, 1, 1, 1, -1]
```

which means:

```
0 → one discovered cluster  
1 → another discovered cluster  
-1 → noise
```

13. Easy edge cases

Edge case 1: Everything becomes noise

Imagine choosing an extremely small eps.

```
•      •      •      •
```

Each point's allowed neighborhood may be so small that almost nobody has enough neighbors.

The result could be:

```
-1  
-1  
-1  
-1
```

meaning all points were classified as noise.

This can be a sign that your neighborhood settings are too restrictive for the data.

Edge case 2: Everything becomes one cluster

Now imagine making eps extremely large.

Points that previously looked separate may become connected:

```
• • • ----- • • • ----- •
```

DBSCAN could treat nearly everything as:

```
Cluster 0
```

Instead of finding meaningful separate groups.

So:

eps too small
→ many points may become noise

eps too large
→ separate groups may merge

The same general idea applies to `min_samples`: changing it changes how easily an area qualifies as dense.

14. Expected interpretation

For today's exercise, your main goal is **not** to calculate exact DBSCAN results manually.

You should be able to look at a simple collection of points and say:

These points are packed closely together.
→ possible cluster

These other points form another dense region.
→ possible second cluster

and:

This point is far from everything else.
→ possible noise

The key lesson is:

DBSCAN discovers clusters by finding connected dense regions instead of requiring a fixed number of clusters beforehand.

A useful comparison to remember is:

K-Means
"How many clusters should I create?"

DBSCAN
"Where are the dense groups?"

15. Hint only

For the problem in Section 9, first ignore DBSCAN terminology entirely.

Just look at:

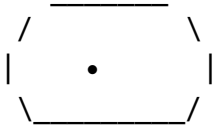
A, B, C

D, E, F

G

Ask yourself which coordinates are very similar to one another.

Then imagine drawing a **small neighborhood circle** around every point:



Consider whether that circle would contain enough nearby points to make the area seem crowded.

Finally, think about the isolated point separately.

For Day 114, remember this flow:

Nearby points



Enough neighbors?



Dense region



Connect dense regions



Cluster

Not enough neighbors



Possible noise

You do **not** need advanced density mathematics or clustering-quality metrics yet.